

Task1

Task 1

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```
GNU nano 7.2 week4_task1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    for (int i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] == 0) {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);
            printf("Child %d completed. PID = %d\n", i + 1, getpid());
            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");
    printf("Using wait():\n");
    pid_t wpid = wait(&status);
    if (WIFEXITED(status)) {
        printf("wait() detected child PID %d finished with exit status %d\n",
            wpid, WEXITSTATUS(status));
    }
}
```

```
teja@LAPTOP-FGDPB110:~$ nano week4_task1.c
teja@LAPTOP-FGDPB110:~$ nano week4_task1.c
teja@LAPTOP-FGDPB110:~$ gcc week4_task1.c -o week4_task1
teja@LAPTOP-FGDPB110:~$ ./week4_task1
Parent Process Started
Parent PID: 540
Child 1 started. PID = 541, Parent PID = 540
Child 2 started. PID = 542, Parent PID = 540
All 3 child processes have been created.
Using wait():
Child 3 started. PID = 543, Parent PID = 540
Child 1 completed. PID = 541
wait() detected child PID 541 finished with exit status 10
Using waitpid():
waitpid() detected child PID 541 finished with exit status 10
Child 2 completed. PID = 542
waitpid() detected child PID 542 finished with exit status 11
Child 3 completed. PID = 543
waitpid() detected child PID 543 finished with exit status 12
Parent process completed.
teja@LAPTOP-FGDPB110:~$ |
```

```

Parent Process Started
Parent PID: 540
Child 1 started. PID = 541, Parent PID = 540
Child 2 started. PID = 542, Parent PID = 540
All 3 child processes have been created.
Using wait():
Child 3 started. PID = 543, Parent PID = 540
Child 1 completed. PID = 541
wait() detected child PID 541 finished with exit status 10
Using waitpid():
waitpid() detected child PID 541 finished with exit status 10
Child 2 completed. PID = 542
waitpid() detected child PID 542 finished with exit status 11
Child 3 completed. PID = 543
waitpid() detected child PID 543 finished with exit status 12
Parent process completed.
teja@LAPTOP-FGDPB110:~$ ps -el
 F S      UID        PID        PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
 4 S      0          1          0  0  80   0 -  5465 -          ?           00:00:01 systemd
 5 S      0          2          1  0  80   0 -   795 -        hvcb0    00:00:00 init-systemd(Ub
 0 S      0          6          2  0  80   0 -   799 -        hvcb0    00:00:00 init
 4 S      0         58          1  0  79  -1 - 12592 -          ?           00:00:00 systemd-journal
 4 S      0        109          1  0  80   0 -  6321 -          ?           00:00:00 systemd-udevd
 4 S     991        123          1  0  80   0 -  5336 -          ?           00:00:00 systemd-resolve
 4 S     996        124          1  0  80   0 - 22759 -          ?           00:00:00 systemd-timesyn
 4 S      0        187          1  0  80   0 -  1061 -          ?           00:00:00 cron
 4 S     101        188          1  0  80   0 -  2387 -          ?           00:00:00 dbus-daemon
 4 S      0        202          1  0  80   0 -  4495 -          ?           00:00:00 systemd-logind
 4 S      0        204          1  0  80   0 - 439157 -          ?           00:00:00 wsl-pro-service
 4 S      0        226          1  0  80   0 -   781 -        tty1     00:00:00 agetty
 4 S     102        227          1  0  80   0 -  55629 -          ?           00:00:00 rsyslogd
 4 S      0        234          1  0  80   0 - 26759 -          ?           00:00:00 unattended-upgr
 5 S      0        336          2  0  80   0 -   796 -          ?           00:00:00 SessionLeader
 5 S      0        337        336  0  80   0 -   800 -          ?           00:00:00 Relay(339)
 4 S    1000        339        337  0  80   0 -  1520 do_wai pts/0     00:00:00 bash
 4 S      0        340          2  0  80   0 -  1676 -          pts/1     00:00:00 login
 4 S    1000        390          1  0  80   0 -  5081 -          ?           00:00:00 systemd
 5 S    1000        391        390  0  80   0 -  5292 -          ?           00:00:00 (sd-pam)
 4 S    1000        415        340  0  80   0 -  1520 poll_s pts/1     00:00:00 bash
 0 R    1000        563        339  50  80   0 -  2080 -          pts/0     00:00:00 ps
teja@LAPTOP-FGDPB110:~$ |

```

```

 0 R    1000        563        339  50  80   0
teja@LAPTOP-FGDPB110:~$ ps
  PID TTY          TIME CMD
  339 pts/0      00:00:00 bash
  568 pts/0      00:00:00 ps
teja@LAPTOP-FGDPB110:~$ |

```

```
41 free
42 ps
43 top
44 vmstat
45 lsblk
46 lscpu
47 mount
48 history
49 clear
50 bc
51 echo hi
52 factor 456
53 yes
54 mkdir OSSP
55 cd OSSP
56 git init
57 mkdir src include obj bin docs tests scripts assets
58 touch .gitignore README.md LICENSE Makefile
59 touch src/main.c
60 touch src/shell.c
61 touch src/parser.c
62 touch src/executor.c
63 touch src/builtin.c
64 touch include/shell.h
65 touch include/parser.h
66 touch include/executor.h
67 touch include/builtin.h
68 touch docs/design.md
69 touch docs/report.pdf
70 touch tests/test_parser.c
71 touch tests/test_executor.c
72 touch scripts/run.sh
73 touch assets/architecture.png
74 tree
75 sudo apt install tree
76 tree
77 cd ~/OSSP
78 nano command_executor.c
79 gcc command_executor.c -o command_executor
80 ./command_executor
81 gcc process.c -o process
```

```

81 gcc process.c -o process
82 ./process
83 gcc process.c -o process
84 gcc --version
85 nano process.c
86 gcc process.c -o process
87 ./process
88 ls
89 pwd
90 date
91 cal
92 whoami
93 hostname
94 uname
95 uname -a
96 ps
97 history
98 lscpu
99 ps
100 ps -ef
101 history
102 mkdir OSSP
103 cd OSSP
104 mkdir OSSP
105 cd OSSP
106 git init
107 mkdir src include obj bin docs tests scripts assets
108 touch src/main.c
109 touch src/shell.c
110 touch src/parser.c
111 touch src/executor.c
112 touch src/builtin.c
113 Install MongoDB Compass (Standalone)
114 cd ~
115 mkdir week4

```

A parent process can create multiple child processes using the `fork()` system call. After creating the child processes, the parent may need to wait for them to complete before continuing.

The `wait()` function allows the parent process to wait until any one child process finishes. The `waitpid()` function gives more control because the parent can wait for a specific child process.

In this practical, three child processes are created. The parent uses `wait()` and `waitpid()` to synchronize their completion and identify when each child process has finished

Task 2 Question

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Parent sleeping...\n");

        sleep(10);

        wait(NULL);

        printf("Parent terminating...\n");
    }

    return 0;
}

```

```

teja@LAPTOP-FGDPB110:~$ nano week4_task2.c
teja@LAPTOP-FGDPB110:~$ gcc week4_task2.c -o week4_task2
teja@LAPTOP-FGDPB110:~$ ./week4_task2
Parent process: PID = 733
Parent sleeping...
Child process: PID = 734
Child PID = 734
Child process terminating...
ps -elParent terminating...
teja@LAPTOP-FGDPB110:~$ ps -el
 F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
 4 S   0         1         0  0  80   0 -   5465 -          ?         00:00:01 systemd
 5 S   0         2         1  0  80   0 -    795 -          hvvc0       00:00:00 init-systemd(Ub
 0 S   0         6         2  0  80   0 -    799 -          hvvc0       00:00:00 init
 4 S   0        58         1  0  79  -1 - 12592 -          ?         00:00:00 systemd-journal
 4 S   0       109         1  0  80   0 -   6321 -          ?         00:00:00 systemd-udev
 4 S  991       123         1  0  80   0 -   5336 -          ?         00:00:00 systemd-resolve
 4 S  996       124         1  0  80   0 -  22759 -          ?         00:00:00 systemd-timesyn
 4 S   0       187         1  0  80   0 -   1061 -          ?         00:00:00 cron
 4 S  101       188         1  0  80   0 -   2387 -          ?         00:00:00 dbus-daemon
 4 S   0       202         1  0  80   0 -   4495 -          ?         00:00:00 systemd-logind
 4 S   0       226         1  0  80   0 -    781 -          tty1       00:00:00 agetty
 4 S  102       227         1  0  80   0 -  55629 -          ?         00:00:00 rsyslogd
 4 S   0       234         1  0  80   0 -  26759 -          ?         00:00:00 unattended-upgr
 4 S   0       340         2  0  80   0 -   1676 -          pts/1     00:00:00 login
 4 S 1000       390         1  0  80   0 -   5081 -          ?         00:00:00 systemd
 5 S 1000       391       390  0  80   0 -   5292 -          ?         00:00:00 (sd-pam)
 4 S 1000       415       340  0  80   0 -   1520 poll_s pts/1     00:00:00 bash
 5 S   0       700         2  0  80   0 -    796 -          ?         00:00:00 SessionLeader
 5 S   0       702       700  0  80   0 -    800 -          ?         00:00:00 Relay(705)
 4 S 1000       705       702  0  80   0 -   1520 do_wai pts/0     00:00:00 bash
 0 R 1000       737       705  0  80   0 -   2080 -          pts/0     00:00:00 ps
teja@LAPTOP-FGDPB110:~$ ps -el | grep Z
 F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
teja@LAPTOP-FGDPB110:~$ gcc week4_task2.c -o week4_task2
./week4_task2
Parent process: PID = 747
Parent sleeping...
Child process: PID = 748
Child PID = 748
Child process terminating...

```

Theory

A zombie process is a child process that has finished its execution, but its parent process has not yet collected its exit status. Because of this, the child remains in the process table.

In this practical, the child process terminates while the parent process is sleeping. The process table can be checked using the `ps -el` command, where the zombie process appears with the state Z.

To remove the zombie process, the parent uses the `wait()` system call. This collects the child's exit status and allows the system to remove the child from the process tabl